

Learning Notes

Protocols, Concepts & Improvement Ideas from the ESP32-CAM Animal Deterrent Project

A Study Companion

Prepared for: Girdhari Chaturvedi
Diploma - Electronics & Telecommunication (ET&T;)
Focus Area: Embedded Systems / IoT

Why This Document

The Animal Deterrent project (ESP32-CAM + Blynk + Python AI detection) touches many real-world embedded systems and IoT concepts. This document goes beyond "how to build it" and explains the underlying technologies being used, why they matter, and what to learn next to grow as an Embedded/IoT engineer.

1. Protocols & Technologies Used in This Project

1.1 Networking Layer

Protocol / Tech	Where It Is Used	What It Does
WiFi (IEEE 802.11 b/g/n)	ESP32-CAM's connection to the router	Wireless physical/link layer connectivity — lets the ESP32 join the local network
TCP/IP	Underlies every network exchange	Provides reliable, ordered delivery of data between devices; almost all higher-level protocols rely on it
HTTP (HyperText Transfer Protocol)	Used by the web server on port 81 (/, /stream, /capture, /trigger)	Application layer protocol: the browser or Python script sends a GET request to retrieve data
MJPEG (Motion JPEG)	The /stream endpoint	Not a real video codec — it is a sequence of individual JPEG images served over HTTP

1.2 Camera Hardware Interfaces

These are internal to the ESP32-CAM board — they connect the ESP32 chip to the OV2640 camera sensor and are not visible over WiFi, but understanding them explains the pin definitions in the code.

Interface	Purpose	Related Code
SCCB (Serial Camera Control Bus)	An I2C-like protocol used to configure the camera sensor's internal settings	GPIO pins 18 (SDA), 19 (SCL), 22 (GPIO)
DVP (Digital Video Port)	A parallel interface that transfers raw pixel data from the sensor to the processor	GPIO pins 2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 20, 21, 23, 24, 25, 26, 27, 28, 29, 30, 31

1.3 Application / Cloud Layer

Protocol	Where Used	Notes
Blynk's proprietary protocol	Blynk.begin() / Blynk.run() / Blynk.logEvent()	A lightweight, custom TCP-based protocol built specifically for IoT devices
SMTP (Simple Mail Transfer Protocol)	Used by SMTPClient, smtplib.SMTP('smtp.gmail.com', 587)	Standard email protocol for sending email. STARTTLS upgrade is required for TLS encryption
UART (Universal Asynchronous Receiver/Transmitter)	Used by Serial.begin(), Serial.print(), Serial.write()	A simple two-wire (TX/RX) serial protocol; used only over USB for debugging

2. Making the Project Better

These are realistic next steps, grouped by theme, that would take this from a "working prototype" to a more robust, production-style IoT system.

2.1 Power Efficiency

Currently the Python script polls the ESP32-CAM for a new photo every 5 seconds, whether or not anything is actually happening — this wastes bandwidth, PC resources, and (if the ESP32 were battery-powered) battery life.

- **PIR motion sensor** — add a passive infrared sensor so the camera is only actively used when motion is physically detected, instead of constant polling.
- **Deep Sleep mode** — put the ESP32 into a low-power sleep state between detections; it wakes up on an interrupt (e.g. the PIR sensor pin going HIGH). Can extend battery life from hours to days/weeks.
- **Solar + battery pack** — combined with deep sleep, makes the whole unit deployable in a field with no wired power at all.

2.2 Security

The current version prioritises "getting it working" over security — reasonable for a learning project, but worth understanding for anything that will actually be deployed.

- **WiFiManager library** — instead of hardcoding the WiFi SSID/password directly in the source code, this library creates a temporary setup WiFi network (a "captive portal") the first time the device boots, letting you enter credentials from a phone browser. Safer and more flexible.
- **Authentication on endpoints** — right now, anyone on the same WiFi network can open /trigger or view the stream. Adding a simple shared secret/token check in the HTTP handler prevents that.
- **HTTPS/TLS** — the project currently uses plain HTTP, so anyone capturing network traffic could see the video stream. HTTPS encrypts this, at the cost of extra complexity and processing overhead on the ESP32.

2.3 On-Device (Edge) AI

Currently, all the "intelligence" (YOLO object detection) runs on a PC because the ESP32 alone is too weak for it. Moving some of that onto the ESP32 itself is a natural next challenge.

- **ESP-WHO** — Espressif's own library with lightweight face detection/recognition models designed to run directly on ESP32/ESP32-CAM hardware.
- **Edge Impulse** — a free platform for training and exporting small, optimised AI models (TensorFlow Lite Micro) that can run on microcontrollers, without needing deep ML expertise.
- **Trade-off to understand:** On-device AI removes the dependency on a PC always being on, but the models must be much smaller and simpler, so accuracy is generally lower than a full YOLO model running on a PC.

2.4 Smarter Features

- **Face recognition** — extend detection to tell apart known family members from an unfamiliar person, instead of alerting on every person equally.

- **SD card logging** — store captured photos and event logs locally, so data is not lost if WiFi or the internet connection drops.
- **Blynk History Graph widget** — visualise past alerts over time as a dashboard, instead of only getting one-off push notifications.

3. Core Concepts Worth Learning Next

These are broader Embedded/IoT concepts that this project touches on. Understanding them well is valuable for interviews, further projects, and general engineering judgement — not just for this specific system.

3.1 Edge Computing vs Cloud Computing

This project actually demonstrates a hybrid of both. The ESP32-CAM (an "edge" device) does the simple, cheap jobs — capturing images and serving a stream. The PC (acting like a small local "cloud"/server) does the heavy computation — running the YOLO AI model. This split is extremely common in real IoT systems: edge devices are cheap and low-power but limited; cloud/servers are powerful but add cost, latency, and a dependency on connectivity.

3.2 Polling vs Event-Driven Architecture

The Python script currently "polls" — it repeatedly asks "is there anything new?" every 5 seconds, regardless of whether anything has actually changed. An event-driven design instead reacts only when something actually happens (e.g. a PIR sensor interrupt firing). Event-driven systems are generally more power-efficient and responsive, and this distinction is a common interview topic in embedded/IoT roles.

Aspect	Polling (current)	Event-Driven (better)
When action happens	On a fixed timer, regardless of real activity	Only when a real trigger occurs
Power/resource use	Wasted cycles when nothing is happening	Efficient — idle until triggered
Responsiveness	Delayed by up to the polling interval	Near-instant reaction

3.3 RTOS (Real-Time Operating System)

The ESP32 actually runs on **FreeRTOS** under the hood, even though the Arduino IDE hides this detail. When the code appears to run `Blynk.run()` and camera streaming "at the same time" inside `loop()`, what is really happening involves multiple underlying tasks being scheduled by FreeRTOS. Understanding RTOS concepts — tasks, priorities, scheduling — becomes important for more advanced embedded work.

3.4 OTA (Over-The-Air) Updates

Right now, updating the code requires physically connecting the FTDI programmer every time. Production IoT devices typically support OTA updates — new firmware is pushed over WiFi, with no physical access needed. ESP32 supports this via the `ArduinoOTA` library, and it is a valuable skill to learn for any device that will be deployed somewhere hard to reach.

3.5 MQTT Protocol

Blynk uses its own proprietary protocol, but the most widely used IoT messaging protocol across the industry is **MQTT** (Message Queuing Telemetry Transport). It is lightweight, uses a publish-subscribe model (devices publish to a "topic"; other devices/services subscribe to it), and is supported by almost every IoT platform (AWS IoT, Azure IoT Hub, Home Assistant, etc.) — unlike Blynk's protocol, which only works with Blynk. Learning MQTT is one of the most transferable skills for IoT work.

Aspect	HTTP Polling (used here)	MQTT (industry standard)
Connection style	New request each time, connection closes after each poll	One persistent connection, always open

Communication model	Request-response (client asks, server answers)	Publish-subscribe (broadcast to interested parties)
Efficiency for many devices	Less efficient — overhead per request	Very efficient — designed for many small, frequent messages
Typical use	Simple web pages / occasional data fetch	Sensor networks, smart home, industrial IoT

Suggested exercise: try replacing the Blynk notification in this project with an MQTT broker (e.g. a free broker like HiveMQ or a self-hosted Mosquitto broker) — it is a great hands-on way to learn the protocol using a system you already understand.

4. Suggested Learning Roadmap

A practical order to explore the topics above, building on skills already used in this project:

Step	Topic	Why
1	PIR Sensor Integration	Easiest next step — pure hardware + simple code logic. Immediately makes the project
2	MQTT Basics	Set up a free MQTT broker, publish a simple sensor value from the ESP32, subscribe to
3	Deep Sleep & Power Management	Combine with the PIR sensor to build real battery-powered deployment experience.
4	WiFiManager / Secure Credential Hashing	Small change with a big real-world security lesson.
5	OTA Updates	Once comfortable with the WiFi stack, learn to push firmware updates wirelessly.
6	Edge AI (ESP-WHO / Edge Impulse)	A meaningful jump in difficulty — attempt after the basics above feel comfortable.

Note: This roadmap is a suggestion based on the natural progression of skills — there is no fixed order, and it's fine to explore whichever topic feels most interesting first.

End of Notes